Experiment - 11

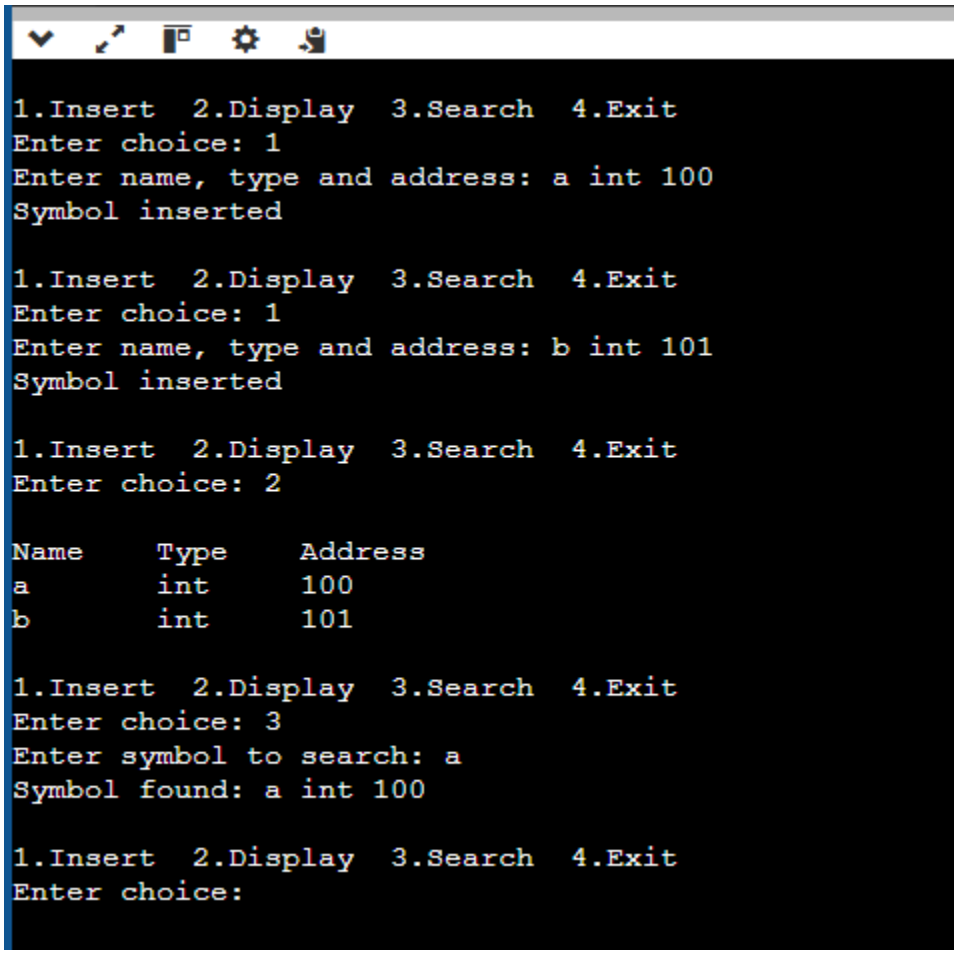
Implement a C program to perform symbol table operations.

-> CODE :

```
#include <stdio.h>
#include <string.h>
struct symbol
{
    char name[20], type[20];
    int address;
};
int main()
{
    struct symbol s[20];
    int n = 0, ch, i;
    char name[20];
    while (1)
    {
        printf("\n1.Insert  2.Display  3.Search  4.Exit\n");
        printf("Enter choice: ");
        if (scanf("%d", &ch) != 1)
        {
            printf("Invalid input. Enter a number.\n");
            return 0;
        }
        switch (ch)
        {
            case 1:
                if (n == 20)
                {
                    printf("Symbol table is full\n");
```

```
            break;
        }
        printf("Enter name, type and address: ");
        scanf("%19s %19s %d",
              s[n].name, s[n].type, &s[n].address);
        n++;
        printf("Symbol inserted\n");
        break;
    case 2:
        printf("\nName\tType\tAddress\n");
        for (i = 0; i < n; i++)
            printf("%s\t%s\t%d\n",
                   s[i].name, s[i].type, s[i].address);
        break;
    case 3:
        printf("Enter symbol to search: ");
        scanf("%19s", name);
        for (i = 0; i < n; i++)
            if (strcmp(s[i].name, name) == 0)
                break;
        if (i < n)
            printf("Symbol found: %s %s %d\n",
                   s[i].name, s[i].type, s[i].address);
        else
            printf("Symbol not found\n");
        break;
    case 4:
        printf("Exiting program\n");
        return 0;
    default:
        printf("Invalid choice. Enter 1, 2, 3 or 4.\n");
    }
  }
}
```

OUTPUT :



Experiment - 12

1. Write a C program to construct recursive descent parsing for the given grammar

   $E \rightarrow TE'$

   $E' \rightarrow +TE' / \in$

   $T \rightarrow FT'$

   $T' \rightarrow *FT' / \in$

   $F \rightarrow (E) /$ id

-> CODE :

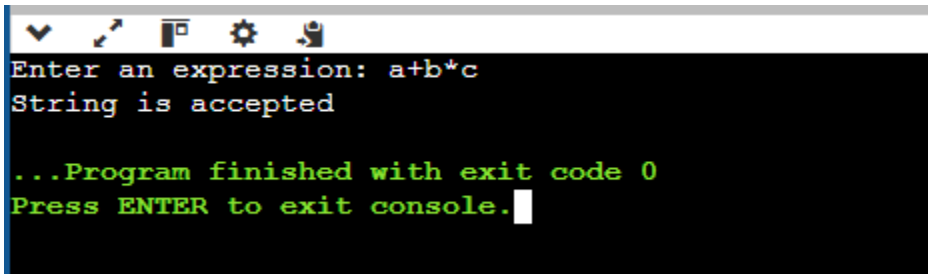
```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
char s[100];
int i = 0;
int E();
int EP();
int T();
int TP();
int F();
int E()
{
   return T() && EP();
}
int EP()
{
   if (s[i] == '+')
   {
      i++;
      return T() && EP();
   }
   return 1;
}
int T()
{
   return F() && TP();
}
int TP()
{
   if (s[i] == '*')
   {
      i++;
      return F() && TP();
```

```
    }
    return 1;
}
int F()
{
    if (s[i] == '(')
    {
        i++;
        if (E() && s[i] == ')')
        {
            i++;
            return 1;
        }
        return 0;
    }
    else if (isalpha(s[i]))
    {
        while (isalnum(s[i]))
            i++;
        return 1;
    }
    return 0;
}

int main()
{
    printf("Enter an expression: ");
    scanf("%99s", s);

    if (E() && s[i] == '\0')
        printf("String is accepted");
    else
        printf("String is rejected");
    return 0;
}
```

-> OUTPUT :



Experiment - 13

All languages have Grammar. When people frame a sentence we usually say whether the sentence is framed as per the rules of the Grammar or Not. Similarly use the same ideology, implement either Top Down parsing technique or Bottom Up Parsing technique to check whether the given input string is satisfying the grammar or not.

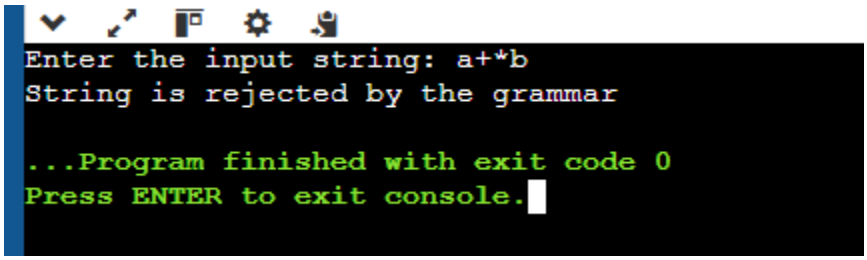
-> CODE :

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>
char s[100];
int i = 0;
void E();
void Ep();
void T();
void Tp();
void F();
void E() {
   T();
   Ep();
}
void Ep() {
   if (s[i] == '+') {
```

```
        i++;
        T();
        Ep();
    }
}
void T() {
    F();
    Tp();
}
void Tp() {
    if (s[i] == '*') {
        i++;
        F();
        Tp();
    }
}
void F() {
    if (isalnum(s[i])) {
        i++;
    }
    else if (s[i] == '(') {
        i++;
        E();
        if (s[i] == ')')
            i++;
        else {
            printf("Invalid string\n");
            exit(0);
        }
    }
    else {
        printf("Invalid string\n");
        exit(0);
    }
}
```

```
int main() {
   printf("Enter an expression: ");
   scanf("%99s", s);
   E();
   if (s[i] == '\0')
      printf("Valid string\n");
   else
      printf("Invalid string\n");
   return 0;
}
```

OUTPUT :



Experiment - 14

The main function of the Intermediate code generation is producing three address code statements for a given input expression. The three address codes help in determining the sequence in which operations are actioned by the compiler. The key work of Intermediate code generators is to simplify the process of Code generators. Write a C Program to Generate the Three address code representation for the given input statement.

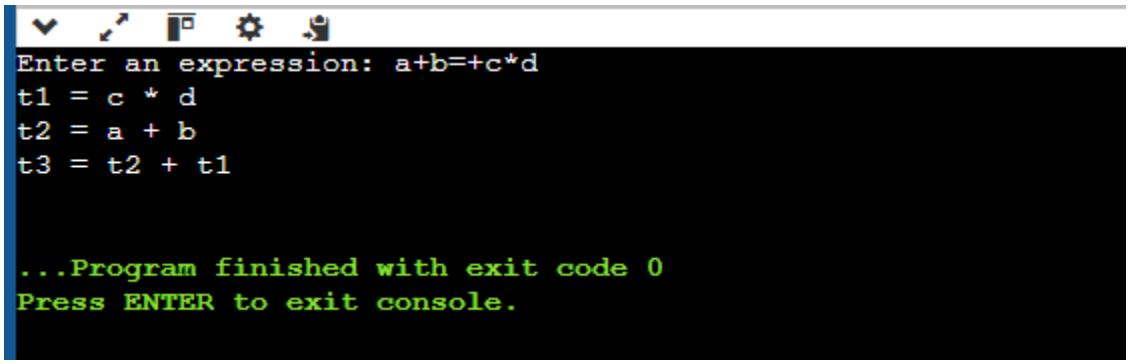
-> CODE :

```
#include <stdio.h>
#include <ctype.h>
#include <string.h>
int main() {
   char s[100], op[100], a[100][20];
```

```
int i, j = 0, k = 0, t = 1;
printf("Enter an expression: ");
scanf("%99s", s);
for (i = 0; s[i]; i++) {
    if (isalnum(s[i])) {
        a[k][0] = s[i];
        a[k++][1] = '\0';
    }
    else if (s[i] == '*' || s[i] == '/') {
        op[j++] = s[i];
        op[j] = '\0';
    }
    else if (s[i] == '+' || s[i] == '-') {
        op[j++] = s[i];
        op[j] = '\0';
    }
}
for (i = 0; i < j; i++) {
    if (op[i] == '*' || op[i] == '/') {
        printf("t%d = %s %c %s\n", t, a[i], op[i], a[i+1]);
        sprintf(a[i], "t%d", t++);
        for (k = i; k < j-1; k++) {
            a[k+1][0] = a[k+2][0];
            a[k+1][1] = '\0';
            op[k] = op[k+1];
        }
        j--;
        i--;
    }
}
while (j > 0) {
    printf("t%d = %s %c %s\n", t, a[0], op[0], a[1]);
    sprintf(a[0], "t%d", t++);
    for (i = 0; i < j-1; i++) {
        strcpy(a[i+1], a[i+2]);
```

```
            op[i] = op[i+1];
        }
        j--;
    }
    return 0;
}
```

-> OUTPUT :



Experiment - 15

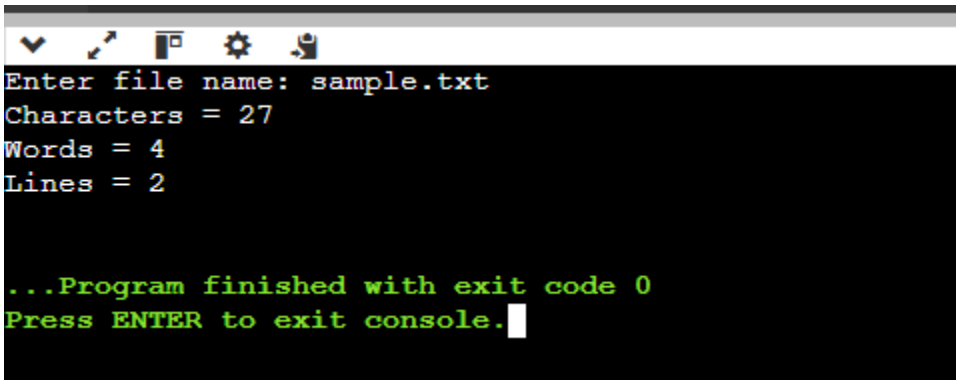
Write a C program for implementing a Lexical Analyzer to Scan and Count the number of characters, words, and lines in a file

-> CODE :

```
#include <stdio.h>
#include <ctype.h>
int main() {
    FILE *fp;
    char file[100];
    int ch, characters = 0, words = 0, lines = 0;
    int inword = 0, last = '\n';
    printf("Enter file name: ");
    scanf("%99s", file);
    fp = fopen(file, "r");
```

```
    if (fp == NULL) {
        printf("File not found\n");
        return 1;
    }
    while ((ch = fgetc(fp)) != EOF) {
        characters++;
        if (ch == '\n')
            lines++;
        if (isspace(ch))
            inword = 0;
        else if (!inword) {
            words++;
            inword = 1;
        }
        last = ch;
    }
    if (characters > 0 && last != '\n')
        lines++;
    fclose(fp);
    printf("Characters = %d\n", characters);
    printf("Words = %d\n", words);
    printf("Lines = %d\n", lines);
    return 0;
}
```

-> OUTPUT :



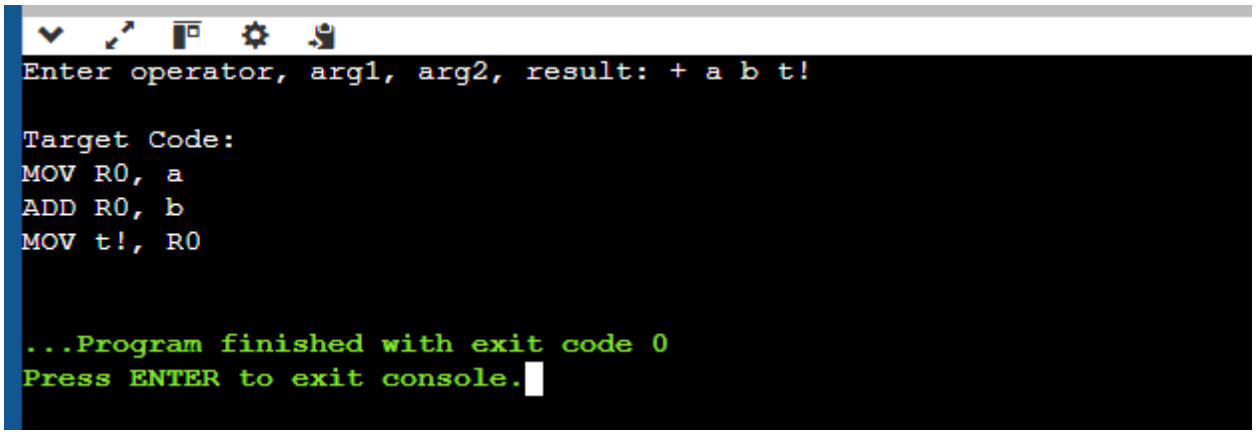
Experiment - 16

Write a C program to implement the back end of the compiler.

-> CODE :

```
#include <stdio.h>
#include <string.h>
int main() {
   char op[10], a[20], b[20], res[20];
   printf("Enter operator, arg1, arg2, result: ");
   fflush(stdout);
   if (scanf("%9s %19s %19s %19s", op, a, b, res) != 4) {
      printf("Invalid input\n");
      return 1;
   }
   printf("\nTarget Code:\n");
   if (strcmp(op, "+") == 0)
      printf("MOV R0, %s\nADD R0, %s\n", a, b);
   else if (strcmp(op, "-") == 0)
      printf("MOV R0, %s\nSUB R0, %s\n", a, b);
   else if (strcmp(op, "*") == 0)
      printf("MOV R0, %s\nMUL R0, %s\n", a, b);
   else if (strcmp(op, "/") == 0)
      printf("MOV R0, %s\nDIV R0, %s\n", a, b);
   else {
      printf("Invalid operator\n");
      return 1;
   }
   printf("MOV %s, R0\n", res);
   return 0;
}
```

-> OUTPUT :



Experiment - 17

Write a C program to compute LEADING( ) – operator precedence parser for the given grammar

E → E + T | T

T → T * F | F

F → (E) | id

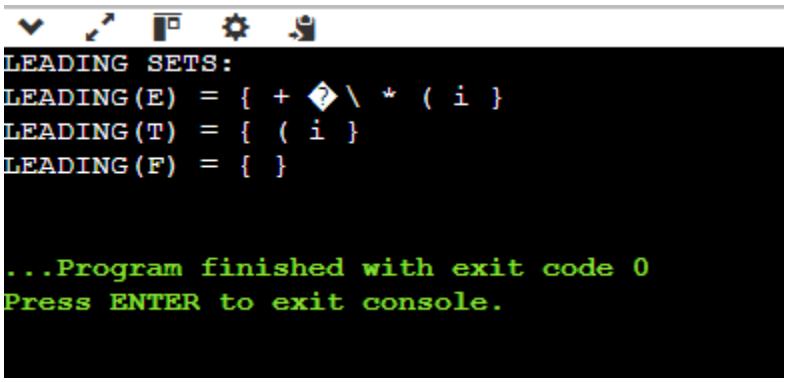
-> CODE :

```
#include <stdio.h>
#include <string.h>
int main() {
    char nt[] = {'E', 'T', 'F'};
    char leading[3][20] = {{0}};
    char productions[6][10] = {
        "E+T", "T", "T*F", "F", "(E)", "id"
    };
    char lhs[] = {'E', 'E', 'T', 'T', 'F', 'F'};
    int i, j, k, changed, n = 0;
    do {
        changed = 0;
        for (i = 0; i < 6; i++) {
            int p = lhs[i] - 'E';
```

```
    if (productions[i][0] == '(' ||
        productions[i][0] == 'i' ||
        productions[i][0] == '+' ||
        productions[i][0] == '*') {
        char c = productions[i][0];
        if (!strchr(leading[p], c)) {
            int len = strlen(leading[p]);
            leading[p][len] = c;
            leading[p][len + 1] = '\0';
            changed = 1;
        }
    }
    else {
        int q = productions[i][0] - 'E';
        for (j = 0; leading[q][j]; j++) {
            if (!strchr(leading[p], leading[q][j])) {
                int len = strlen(leading[p]);
                leading[p][len] = leading[q][j];
                leading[p][len + 1] = '\0';
                changed = 1;
            }
        }
        if (productions[i][1] == '+' ||
            productions[i][1] == '*') {
            char c = productions[i][1];
            if (!strchr(leading[p], c)) {
                int len = strlen(leading[p]);
                leading[p][len] = c;
                leading[p][len + 1] = '\0';
                changed = 1;
            }
        }
    }
}
```

```
    }
while (changed);
    printf("LEADING SETS:\n");
    for (i = 0; i < 3; i++) {
        printf("LEADING(%c) = { ", nt[i]);
        for (j = 0; leading[i][j]; j++)
            printf("%c ", leading[i][j]);
        printf("}\n");
    }
    return 0;
}
```

-> OUTPUT :



Experiment - 18

Write a C program to compute TRAILING( ) – operator precedence parser for the given grammar

E → E + T | T
T → T * F | F
F → (E) | id

-> CODE :

```
#include <stdio.h>
#include <string.h>
```

```
int main() {
    char nt[] = {'E', 'T', 'F'};
    char trailing[3][20] = {{0}};
    char productions[6][10] = {
        "E+T", "T", "T*F", "F", "(E)", "id"
    };
    char lhs[] = {'E', 'E', 'T', 'T', 'F', 'F'};
    int i, j, changed;
    do {
        changed = 0;
        for (i = 0; i < 6; i++) {
            int p = lhs[i] - 'E';
            int len = strlen(productions[i]);
            char c = productions[i][len - 1];
            if (c == ')' || c == 'd') {
                if (!strchr(trailing[p], c)) {
                    int n = strlen(trailing[p]);
                    trailing[p][n] = c;
                    trailing[p][n + 1] = '\0';
                    changed = 1;
                }
            }
            else if (c >= 'E' && c <= 'F') {
                int q = c - 'E';
                for (j = 0; trailing[q][j]; j++) {
                    if (!strchr(trailing[p], trailing[q][j])) {
                        int n = strlen(trailing[p]);
                        trailing[p][n] = trailing[q][j];
                        trailing[p][n + 1] = '\0';
                        changed = 1;
                    }
                }
                if (len > 1 &&
                    (productions[i][len - 2] == '+' ||
                     productions[i][len - 2] == '*')) {
```

```
                c = productions[i][len - 2];
                if (!strchr(trailing[p], c)) {
                    int n = strlen(trailing[p]);
                    trailing[p][n] = c;
                    trailing[p][n + 1] = '\0';
                    changed = 1;
                }
            }
        }
    }
    } while (changed);
    printf("TRAILING SETS:\n");
    for (i = 0; i < 3; i++) {
        printf("TRAILING(%c) = { ", nt[i]);
        for (j = 0; trailing[i][j]; j++) {
            if (trailing[i][j] == 'd')
                printf("id ");
            else
                printf("%c ", trailing[i][j]);
        }
        printf("}\n");
    }
    return 0;
}
```

-> OUTPUT :

